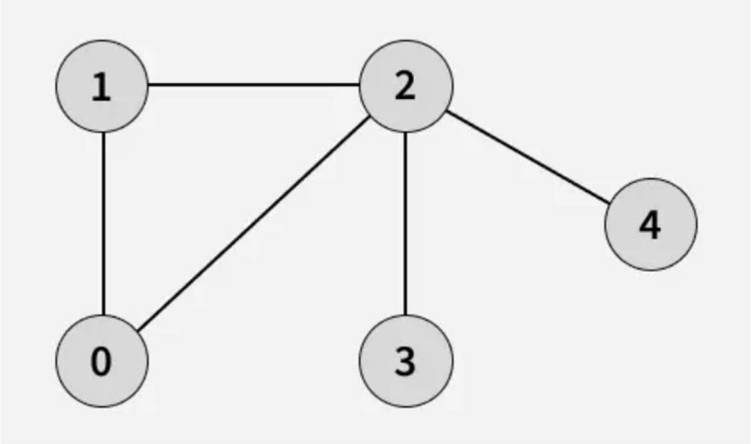
Lab Manual 14

Objectives:

- Understand the basic concept and working mechanism of BFS.
- Implement BFS using a queue and adjacency list representation.
- Apply BFS for shortest path and other real world graph problems.

Breadth-First Search (BFS) is a fundamental graph traversal algorithm used to explore nodes level by level, starting from a selected source node. It uses a queue to visit all immediate neighbors first before moving to the next layer of nodes, making it ideal for finding the shortest path in unweighted graphs and for analyzing connectivity in networks. Because of its systematic and predictable traversal pattern, BFS forms the basis of many real-world applications such as route planning, social network analysis, web crawling, and solving grid-based problems like mazes.



Input: adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]
Output: [0, 1, 2, 3, 4]

Explanation:
Starting from 0, the BFS traversal proceeds as follows:
Visit 0 - Print: 0
Visit 1 (neighbor of 0) - Print: 1
Visit 2 (next neighbor of 0) - Print: 2
Visit 3 (first neighbor of 2 that hasn't been visited yet) - Print: 3
Visit 4 (next neighbor of 2) - Print: 4

Implementation Notes & Common Pitfalls

- **Visited array is essential:** Without it, you can re-enqueue nodes and loop forever on cycles.
- **Queue semantics matter:** Use a FIFO queue (std::queue in C++, deque or collections.deque in Python).
- **Adjacency list vs adjacency matrix:** Use adjacency lists for sparse graphs (faster: O(V+E)), matrices make edge iteration $O(V^2)$.
- **Parent tracking:** Store parent[v] to reconstruct the shortest path. To get path from s to t, follow parents from t back to s and reverse.
- **Grid graphs:** Represent positions as nodes and neighbors as adjacent cells, beware boundaries and visited marking.

BFS Applications

Breadth-First Search is not only a fundamental graph traversal technique but also a versatile tool in solving various computational problems. Its systematic, level-by-level exploration ensures efficiency and accuracy in many scenarios, making it highly useful in both theoretical and practical contexts. BFS has numerous applications across computer science and real-world problems, ranging from pathfinding and network analysis to tree traversals and puzzle solving. The following sections highlight some of the key applications where BFS plays a crucial role.

Shortest Path Finding

Breadth-First Search is widely used to find the **shortest path in an unweighted graph**, where every edge has the same cost or weight (usually considered as 1). Because BFS explores the graph level by level, the first time it reaches a node, it is guaranteed to have found the path with the **minimum number of edges** from the source to that node. This makes BFS an ideal choice for shortest-path problems in scenarios such as communication networks, road maps without varying distances, and grid or maze navigation. By maintaining a parent or predecessor array during traversal, BFS can also reconstruct the exact shortest path from the source to any destination node, making it both efficient and easy to implement for practical shortest path tasks.

```
void BFSShortestPath(int V, int source, int target) {
    bool visited[100] = { false };
    int distance[100];
    int parent[100];

    for (int i = 0; i < V; i++) {
        distance[i] = -1;
        parent[i] = -1;
    }

    queue<int> q;
    visited[source] = true;
    distance[source] = 0;
    q.push(source);

    while (!q.empty()) {
        int u = q.front(); q.pop();

        for (int i = 0; i < degree[u]; i++) {
            int v = adj[u][i];
            if (!visited[v]) {
                visited[v] = true;
                distance[v] = distance[u] + 1;
                parent[v] = u;
                q.push(v);
```

```
            }
        }
    }

    if (!visited[target]) {
        cout << "No path from " << source << " to " << target << endl;
        return;
    }

    stack<int> path;
    for (int at = target; at != -1; at = parent[at])
        path.push(at);

    cout << "Shortest path length: " << distance[target] << endl;
    cout << "Path: ";
    while (!path.empty()) {
        cout << path.top() << " ";
        path.pop();
    }
    cout << endl;
}
```

Cycle Detection in Graphs

A **cycle** in a graph occurs when a path starts from a vertex and comes back to the same vertex without repeating any edge. BFS can be used to detect cycles in both **undirected** and **directed graphs**, although the approach differs slightly.

```
bool hasCycleUndirected(int V) {
    bool visited[100] = {false};
    int parent[100];

    for (int i = 0; i < V; i++) {
        if (visited[i]) continue;

        queue<int> q;
        q.push(i);
        visited[i] = true;
        parent[i] = -1;

        while (!q.empty()) {
            int u = q.front(); q.pop();

            for (int j = 0; j < degree[u]; j++) {
```

```
                int v = adj[u][j];

                if (!visited[v]) {
                    visited[v] = true;
                    parent[v] = u;
                    q.push(v);
                }
                else if (v != parent[u]) {
                    return true;
                }
            }
        }
    }
    return false;
}
```

```
bool hasCycleDirected(int V) {
    queue<int> q;
    int count = 0;

    for (int i = 0; i < V; i++)
        if (indeg[i] == 0)
            q.push(i);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        count++;

        for (int j = 0; j < degree[u]; j++) {
            int v = adj[u][j];
            if (--indeg[v] == 0)
                q.push(v);
        }
    }
    return count != V;
}
```

In-Lab Tasks

TASK 1 — Nobita's Race to Shizuka Through Tokyo Town

Tokyo Town is represented as a grid where each cell marks a small area Nobita can move through. Some areas are **free (0)**, while others are **blocked (1)** because Gian has set up traps or is standing there to catch Nobita.

Nobita starts at cell **S**, and Shizuka is waiting for him at cell **T**. Nobita must reach her using the safest and shortest route possible without getting caught by Gian or stepping into any of his traps.

Your task is to implement BFS to help Nobita find this shortest path.

1. BFS with 4-Direction Movement (Nobita Moves Normally Through Town)

Write a Breadth-First Search (BFS) algorithm that calculates the **minimum number of steps** Nobita needs to reach Shizuka **using only the four primary directions**:

- Up
- Down
- Left
- Right

While implementing BFS, ensure that your logic:

- Stays within grid boundaries
- Avoids Gian's traps (blocked cells)
- Prevents revisiting previously explored cells
- Correctly counts the number of steps from S to T

Your program should output:
"Shortest path for Nobita to reach Shizuka using 4 directions: X steps"

2. BFS with 8-Direction Movement (Nobita Uses Doraemon's Gadget for Diagonal Moves)

Now Nobita borrows a gadget from Doraemon, such as the **Diagonal Step Enhancer** which allows him to make diagonal movements.

Modify your BFS so that Nobita can move in **eight directions**:

- Up, Down, Left, Right
- Up-Left, Up-Right
- Down-Left, Down-Right

Using this improved movement, compute the new **minimum steps** needed to reach Shizuka.

Your program should output:
"Shortest path for Nobita to reach Shizuka using 8 directions: Y steps"

TASK 2 — Cycle Detection in the "Tokyo Town Road Network"

Tokyo Town from *Doraemon* can be modeled as an undirected graph where each major location like **Nobita's House, Doraemon's Shed, Shizuka's Home, School, Empty Ground, Gian's House and Suneo's Mansion** represents a node. These locations are connected through two-way streets, shortcuts and hidden alleys commonly used by the characters, leading to the possibility of cycles in the road network.

Your task is to implement **Cycle Detection using BFS** on this graph.

You must complete the following coding tasks:

1. **Represent Tokyo Town as an undirected graph** using adjacency lists, assigning each location a node number (e.g., 0–6).
2. **Add edges** between nodes to represent real paths, such as:

   - Nobita's House ↔ Doraemon's Shed
   - Doraemon's Shed ↔ School
   - School ↔ Shizuka's Home
   - Shizuka's Home ↔ Suneo's Mansion
   - Suneo's Mansion ↔ Empty Ground
   - Empty Ground ↔ School
   - Nobita's House ↔ Gian's House
     These connections must ensure the graph contains at least **two cycles**.

3. **Implement BFS from any starting node**, such as Nobita's House, storing a parent array to track the parent of each visited node.
4. **During BFS**, check for cycle conditions:

   - When visiting neighbors, if a neighbor is **already visited** and **not the parent**, a cycle exists.

5. **As soon as a cycle is detected**, print:

    - A message indicating a cycle is present
    - The nodes involved (simple detection is enough)

6. **Write a function bool hasCycleBFS(...)** that returns true if the Tokyo Town graph contains a cycle, otherwise false.
7. **In the main function**, build the Tokyo Town graph, run the cycle detection function, and display the result.

Submission Guidelines

1. submit following the only file strictly following the naming convention
    a. l241234.cpp